

Python

A 4-page quick reference guide

1. Python Fundamentals

Python is a high-level, dynamically typed, interpreted programming language known for readable syntax and a huge standard library. It supports procedural, object-oriented, and functional styles.

Variables & Data Types

```
x = 10 # int
y = 3.14 # float
name = "Alice" # str
is_valid = True # bool
items = [1, 2, 3] # list
info = {"a": 1} # dict
coords = (1, 2) # tuple
unique = {1, 2, 3} # set
```

Control Flow

```
if x > 5:
    print("big")
elif x == 5:
    print("equal")
else:
    print("small")

for i in range(5):
    print(i)

while x > 0:
    x -= 1
```

Functions

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

square = lambda n: n ** 2
```

2. Data Structures & Comprehensions

Python's built-in collections cover most everyday needs. Comprehensions offer a concise way to build lists, dicts, and sets.

Comprehensions

```
squares = [n**2 for n in range(10)]
evens = [n for n in range(20) if n % 2 == 0]
square_map = {n: n**2 for n in range(5)}
unique_lens = {len(w) for w in ["a", "bb", "ccc"]}
```

Common List Methods

- **append(x)** — add item to end
- **sort(key=..., reverse=...)** — sort in place
- **extend(iterable)** — merge another list in
- **pop(i)** — remove & return item at index

Dictionaries

```
person = {"name": "Bob", "age": 30}
person.get("age", 0)
for key, value in person.items():
    print(key, value)
```

3. Object-Oriented Programming

Classes bundle data and behavior. Python supports inheritance, polymorphism, and special (dunder) methods for operator overloading.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

d = Dog("Rex")
print(d.speak())
```

Key Concepts

- **Inheritance** — subclasses reuse and extend parent behavior
- **Encapsulation** — use underscore prefixes (`_x`, `__x`) for privacy conventions
- **Dunder methods** — `__init__`, `__str__`, `__eq__`, `__len__` customize built-in behavior
- **Properties** — `@property` turns a method into an attribute-like accessor

Error Handling

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
finally:
    print("Done")
```

4. Modules, Files & Best Practices

File I/O

```
with open("data.txt", "r") as f:
    contents = f.read()

with open("out.txt", "w") as f:
    f.write("Hello, file!")
```

Modules & Packages

```
import math
from collections import Counter
from datetime import datetime

print(math.sqrt(16))
```

Virtual Environments & pip

```
python -m venv venv
source venv/bin/activate
pip install requests
```

Best Practices

- Follow PEP 8 style conventions for readable code
- Use list/dict comprehensions over manual loops when clear
- Prefer f-strings for formatting: `f"{value:.2f}"`
- Write docstrings and type hints for maintainability
- Use virtual environments to isolate project dependencies